

[Marcar como hecha](#)

Node.js

Ventajas

Los desarrolladores habituales de aplicaciones web no necesitan trabajar a bajo nivel del protocolo HTTP u otros protocolos. En su lugar, tendemos a ser más productivos, trabajando con interfaces de más alto nivel. Por ejemplo, los programadores de PHP asumen que Apache (u otros servidores HTTP) ya están ahí proporcionando el protocolo HTTP, y que no tienen que implementar la parte del servidor HTTP de la pila. Por el contrario, un programador de Node.js implementa un servidor HTTP al que se adjunta su código de aplicación.

Un lenguaje común para frontend y backend ofrece varias ventajas potenciales:

- El mismo personal de programación puede trabajar en ambos lados.
- El código puede migrarse más fácilmente entre el servidor y el cliente.
- Existen formatos de datos comunes (JSON) entre servidor y cliente.
- Existen herramientas de software comunes para servidor y cliente
- Al escribir aplicaciones web, las plantillas de vista pueden utilizarse en ambos lados.

La granularidad, que es un concepto que se ve en programación orientada a componentes, es también todavía más interesante atendiendo a una arquitectura de microservicios, como la se propugna actualmente. Node está también muy enfocado a este método.

Origen

Node fue creado en 2009 por **Ryan Dahl** programador en ese entonces de la empresa Joyent (dedicada a ofrecer servicios de cómputo en la nube) que a su vez se convirtió en la propietaria de la marca Node.js y la que le daría patrocinio y difusión desde el momento de su creación.

Joyent puso todo su empeño para el desarrollo de Node, sin embargo, al ser una empresa del sector privado y no una comunidad o fundación, los avances de Node comenzaron a ser muy lentos, en comparación de lo que la comunidad solicitaba y que también quería contribuir, por esto crearon el proyecto io.js, comenzando a competir con Joyent de tal modo que al cabo, en 2015 y con la fundación Linux como mediadora, se unen ambos proyectos.

Así node, según wikipedia quedó como: "un entorno en tiempo de ejecución multiplataforma, de código abierto, para la capa del servidor, basado en el lenguaje de programación JavaScript, asíncrono, con E/S de datos en una arquitectura orientada a eventos y basado en el motor V8 de Google. Fue creado con el enfoque de ser útil en la creación de programas de red altamente escalables, como por ejemplo, servidores web"

Funcionamiento gral.

De lo que se trataba, planteado por su creador, es evitar las pausas en ejecución que produciría, por ejemplo una consulta a base de datos muy extensa. Incluso realizándose ésta dentro de hilos de ejecución, su ejecución tendría que estar sujeta a los cambios de contexto de los hilos, de modo que se pueda atender a otras operaciones mientras. Pero el cambio de contexto no es gratuito porque más hilos requieren más memoria por estado de hilo y más tiempo en que la CPU está dedicada a la gestión de hilos cuando hay sobrecarga.

En general veremos que en un contexto de trabajo en Internet, donde no sabemos cuántas peticiones tendremos, los hilos no son una solución que pueda escalarse

Las funciones de retorno (callbacks) disparadas de forma asíncrona desde un bucle de eventos son un modelo de concurrencia mucho más simple: más simple de entender, más simple de implementar, más simple de razonar y más simple de depurar y mantener.

El servidor más simple: El objeto http encapsula el protocolo HTTP, y su método http.createServer crea un servidor web completo, escuchando en el puerto especificado en el método listen.

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World\n');
}).listen(8124, "127.0.0.1");
console.log('Server running at http://127.0.0.1:8124/');
```

La idea entonces que se suele encapsular en las palabras async/await, es que usualmente el resultado no aparece en la siguiente línea de código, sino que aparece en una llamada de retorno

Igual que JS, usamos let con variables de alcance de bloque y const por las inamovibles.

?

Instalación

como npm es el repositorio de facto de node.js, la instalación por paquete de win, incluye el repositorio y otras utilidades. En principio con el intérprete node y el repo es bastante, aunque como node tiene muchas versiones, no es mala idea instalar nvm (node version manager), que es un control de versiones de node que permite cambiar de una a otra entre las que están instaladas. También con nvm install, es sencillo instalar nuevas

```
node --version
```

nos dirá la versión actual.

<https://github.com/nvm-sh/nvm>